

TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

OPERATING SYSTEMS LABORATORIES 5 & 6

The File System with Your Own Hands and Inside the xv6 File System

Student: Lucian-Adrian Gavril

Group: FAF-241

Coordinator: univ. assist., Reițman Patricia

Chișinău, 2026

TABLE OF CONTENTS

ABSTRACT 4

INTRODUCTION 5

I Laboratory 5: The File System with Your Own Hands (Linux) 6

1 LINUX FILESYSTEM TREE AND FILE TYPES 7

1.1 System Root and Type Identification 7

1.2 UsrMerge Architecture 8

1.3 Device Nodes in /dev 8

1.4 Storage Comparison: /var/log vs /usr 8

2 INODES, LINKS, AND SPACE ALLOCATION 9

2.1 Inode Decoupling and Hard Link Creation 9

2.2 Symbolic Link Semantics 10

2.3 Directory Link Count Dynamics 10

2.4 Sparse Files and Allocation Granularity 10

3 PERMISSIONS AND SYSTEM INTROSPECTION 11

3.1 Octal Modes and Owner Evaluation 11

3.2 Umask Arithmetic 11

3.3 Directory Read versus Execute 11

3.4 Setuid and Sticky Bit 12

3.5 Disks, Topology, and /proc/self 12

4 LABORATORY 5 STRETCH PROBLEMS 13

II Laboratory 6: Inside the xv6 File System 14

5 XV6 DIRECTORY STRUCTURE AND TYPE DECODING 15

5.1 Filesystem Type Values 15

5.2 Root Directory and Entry Sizing 15

6 THE MYSTAT UTILITY AND HARD LINK MECHANICS 16

6.1 Implementation of user/mystat.c 16

6.2 Observing Hard Link Operations 16

6.3 Unlinking versus Deallocation 17

6.4 On-Disk Inode Structure 18

7 LIMITS AND DISK ARCHITECTURE 19
7.1 Filename Truncation 19
7.2 Maximum File Capacity 19
7.3 Disk Layout 19
8 RECURSIVE SEARCH: USER/FIND.C AND INODE VERIFICATION 20
8.1 Implementation of user/find.c 20
8.2 Testing and Inode Stretch Resolution 21
CONCLUSIONS 23
REFERENCES 24

ABSTRACT

This combined laboratory report investigates the principles, mechanisms, and internal structures of Unix filesystems across two distinct environments: the production Linux Virtual Filesystem (VFS) in Ubuntu 24.04 LTS (kernel 6.6.87.2 on x86_64) and the educational MIT xv6 operating system running on an emulated RISC-V platform via QEMU 8.2.2.

Part I (Laboratory 5) examines the Linux filesystem from user space under an unprivileged user (`ubuntu`, UID 1000). It covers filesystem hierarchies, UsrMerge namespace unification, device inodes, inode-to-name decoupling, hard and symbolic link semantics, sparse file allocation, Discretionary Access Control (DAC) permission mechanics, and `/proc` introspection, including all four optional stretch problems (magic numbers, setuid auditing, open file descriptor unlinking, and loopback ext4 inode sizing).

Part II (Laboratory 6) dives into the xv6 filesystem internals. It analyzes on-disk directory tables, the structural layout of `struct dinode`, theoretical and physical capacity limits from `kernel/fs.h`, and the separation between directory unlinking in `sys_unlink()` and block deallocation in `iput()` and `itrunc()`. Two user-space programs are implemented in C: `mystat.c` (inode and link inspection) and `find.c` (recursive directory search with the `-n` inode display stretch flag).

Keywords: filesystems, linux, xv6, inodes, links, permissions, directory traversal

INTRODUCTION

This supplementary report documents the verified filesystem experiments across Linux VFS and xv6. The implementation details align with the included xv6 source[1] and the individual Lab 5 and Lab 6 deliverables.

Filesystems are among the foundational abstractions of modern operating systems. They bridge persistent storage media and running processes by transforming raw block sectors into structured, human-navigable namespaces. Under Unix and POSIX semantics, filesystems extend beyond persistent disks: device drivers, interprocess communication pipes, and live kernel statistics all present uniform file descriptor interfaces.

This combined report documents the theoretical and practical investigation of both a full-scale production kernel (Linux) and an educational operating system (xv6):

a) **Laboratory 5 (Track A: Linux Warm-up)**: Direct shell inspection of filesystem types, inode table mechanics, hard and symbolic link lifecycles, space allocation in sparse files, Discretionary Access Control (DAC) permissions, process identification in `/proc`, and four advanced system experiments;

b) **Laboratory 6 (Track B: xv6 Filesystem Internals)**: Code-level tracing and live execution in the xv6 RISC-V kernel, developing `user/mystat.c` and `user/find.c`, evaluating on-disk block and inode layouts, analyzing link count decrement versus block deallocation timing, and exploring structural filesystem limits;

Part I

Laboratory 5: The File System with Your Own Hands (Linux)

1 LINUX FILESYSTEM TREE AND FILE TYPES

1.1 System Root and Type Identification

The leading character of an `ls -ld` listing identifies the inode type in the Linux VFS:

```
$ whoami
ubuntu

$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx   1 root root    7 Apr 22  2024 /bin -> usr/bin
crw-rw-rw-   1 root root 1, 3 Oct 12 15:33 /dev/null
drwxr-xr-x 107 root root 4096 Oct 12 15:38 /etc
-rw-r--r--   1 root root 2430 Oct 12 15:38 /etc/passwd
drwxrwxrwt  10 root root 4096 Oct 12 15:38 /tmp
```

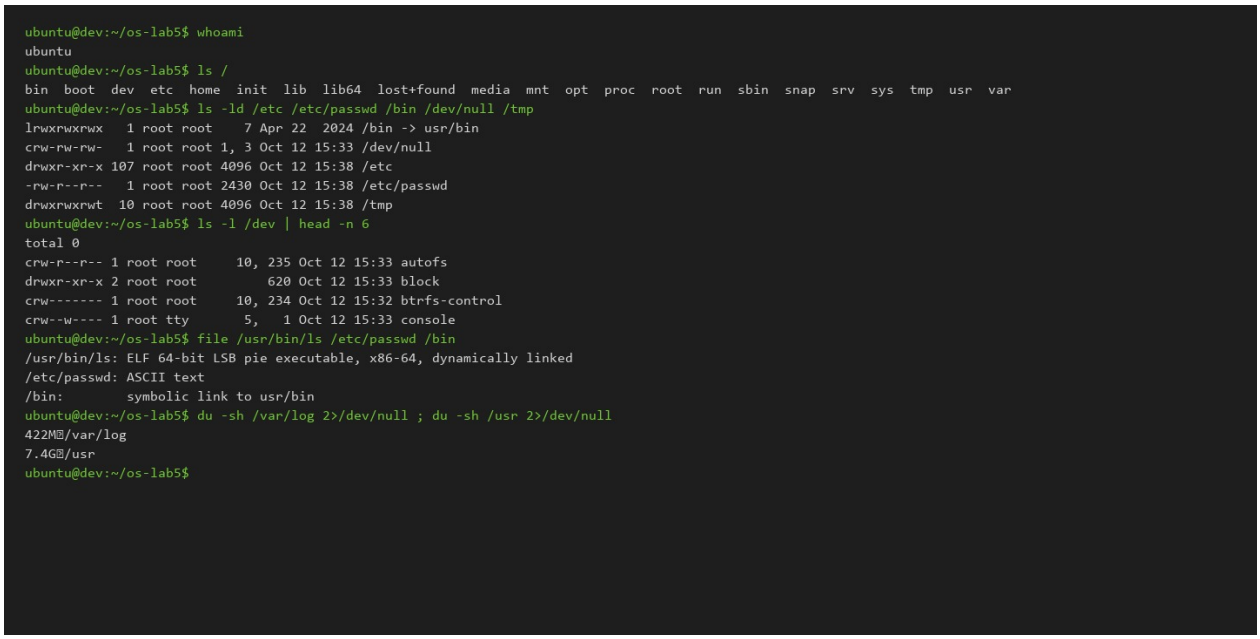


Figure 1.1 – inspecting system root, device nodes, and file types.

Table 1.1 – Observed file types in Ubuntu 24.04 LTS

Path	Type Char	Filesystem Type	Description
`/bin`	`l`	Symbolic Link	Points to `usr/bin` (UsrMerge)
`/dev/null`	`c`	Character Device	Byte-stream sink (major 1, minor 3)
`/etc`	`d`	Directory	System configuration tables
`/etc/passwd`	`-`	Regular File	User database in ASCII text
`/tmp`	`d`	Directory (Sticky)	World-writable temporary workspace

1.2 UsrMerge Architecture

On modern Linux distributions, `/bin` is a symbolic link pointing to `usr/bin`. This design consolidates system software under `/usr`, simplifying package management, snapshotting, and read-only root mounting.

1.3 Device Nodes in /dev

Special device nodes do not consume disk blocks for file content. Instead, their inodes store major and minor numbers:

- **Character Device**: `/dev/null` (major 1, minor 3) and `/dev/console` (major 5, minor 1);
- **Block Device**: `/dev/sda` (major 8, minor 0) and `/dev/sdd` (major 8, minor 48), supporting block-level operations through the page cache;

1.4 Storage Comparison: /var/log vs /usr

```
$ du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
422M    /var/log
7.4G    /usr
```

The `/usr` hierarchy occupies 7.4 GB, which is roughly 7.0 GB larger ($\approx 17.5\times$) than `/var/log` (422 MB). This reflects `/usr`'s role in storing all system binaries, libraries, toolchains, and development headers.

2 INODES, LINKS, AND SPACE ALLOCATION

2.1 Inode Decoupling and Hard Link Creation

Filenames are not stored inside inodes; they reside in directory tables as mapping entries:

```
$ echo hello > notes.txt
$ stat notes.txt
  File: notes.txt
  Size: 6           Blocks: 8          IO Block: 4096   regular file
Device: 8,48    Inode: 18829       Links: 1
```

Creating a hard link creates an additional directory entry referencing the same inode:

```
$ ln notes.txt hard.txt
$ ls -li notes.txt hard.txt
18829 -rw-r--r-- 2 ubuntu ubuntu 6 Oct 12 15:39 hard.txt
18829 -rw-r--r-- 2 ubuntu ubuntu 6 Oct 12 15:39 notes.txt
$ rm notes.txt
$ cat hard.txt
hello
```

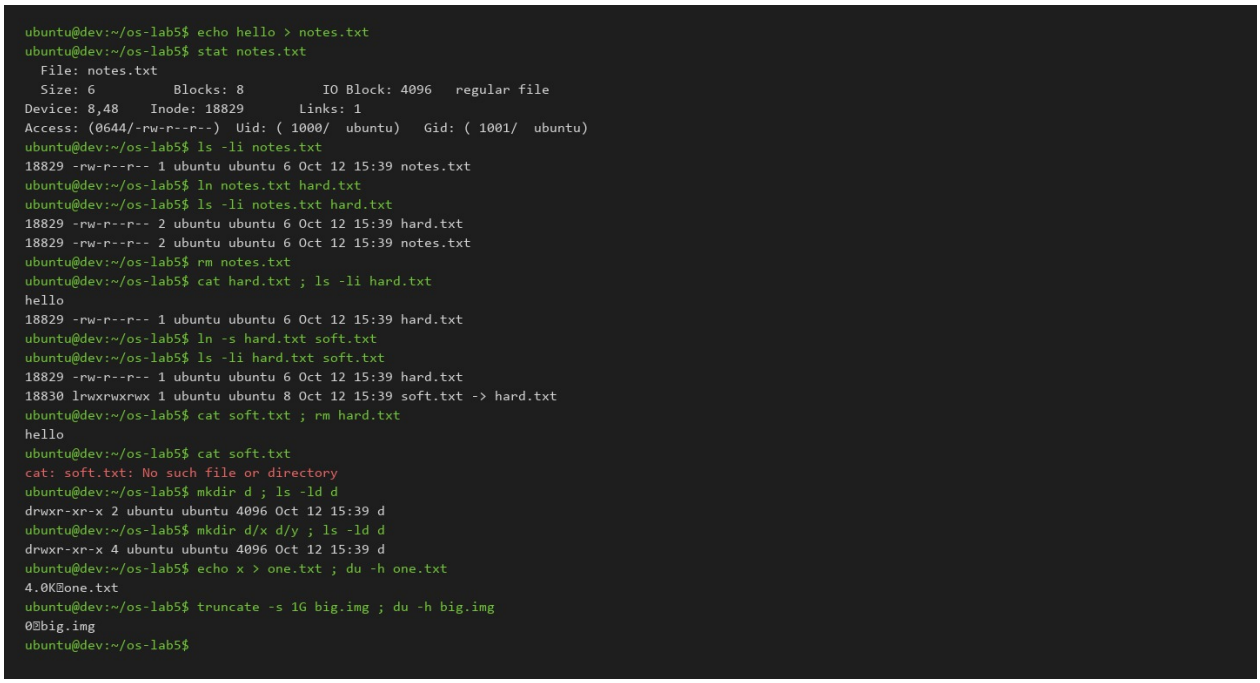


Figure 2.1 – inode inspection, hard links, dangling symlinks, and sparse files.

The `rm` command unlinks the name `notes.txt` and decrements the inode link count from 2 to 1. Because `hard.txt` maintains a reference to inode 18829, the data blocks remain allocated and readable.

2.2 Symbolic Link Semantics

Symbolic links store the path of their target rather than referencing an inode number:

```
$ ln -s hard.txt soft.txt
$ ls -li hard.txt soft.txt
18829 -rw-r--r-- 1 ubuntu ubuntu 6 Oct 12 15:39 hard.txt
18830 lrwxrwxrwx 1 ubuntu ubuntu 8 Oct 12 15:39 soft.txt -> hard.txt
$ rm hard.txt
$ cat soft.txt
cat: soft.txt: No such file or directory
```

Unlinking `hard.txt` leaves `soft.txt` dangling. The exact size of `soft.txt` is 8 bytes, matching the length of the string `"hard.txt"`.

2.3 Directory Link Count Dynamics

```
$ mkdir d ; ls -ld d
drwxr-xr-x 2 ubuntu ubuntu 4096 Oct 12 15:39 d
$ mkdir d/x d/y ; ls -ld d
drwxr-xr-x 4 ubuntu ubuntu 4096 Oct 12 15:39 d
```

Under the classic ext-style directory link model, a directory starts with 2 links: its entry in the parent directory and the self-referencing '.' entry. Each subdirectory adds a '..' entry pointing back to the parent, incrementing the parent's link count to 4. This count is filesystem-dependent; it is not a universal Linux invariant.

2.4 Sparse Files and Allocation Granularity

```
$ echo x > one.txt ; du -h one.txt
4.0K    one.txt
$ truncate -s 1G big.img ; du -h big.img
0       big.img
```

Ext4 allocates space in 4 KiB block increments, requiring 4.0 KiB even for a 2-byte file. In contrast, `truncate` creates a sparse file: it updates logical size metadata without writing data blocks, resulting in 0 physical blocks allocated.

3 PERMISSIONS AND SYSTEM INTROSPECTION

3.1 Octal Modes and Owner Evaluation

```
$ chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied
```

When the accessing user is the file owner, the kernel evaluates only the owner permission bits. Because they are `000`, access is denied. Ownership grants metadata authority (the right to invoke `chmod()`), not an automatic read bypass.

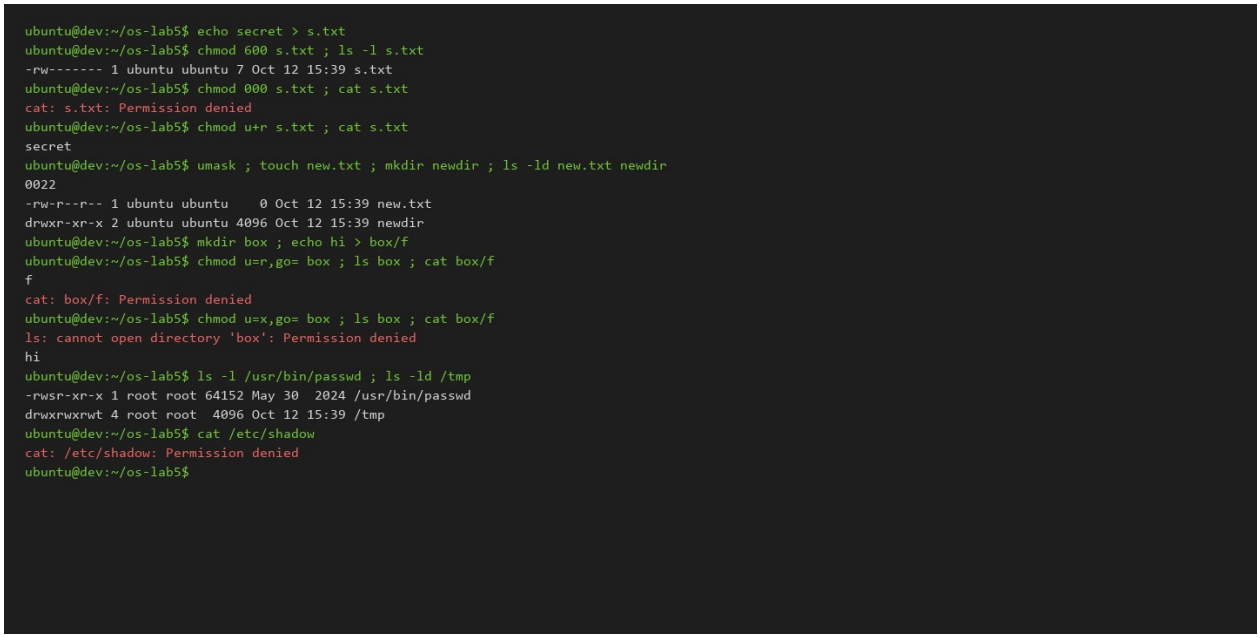


Figure 3.1 – permission evaluation, umask verification, and directory traversal.

3.2 Umask Arithmetic

The observed umask is `0022`. Files default to base mode `0666` ($0666 \ \& \ \sim 0022 = 0644$), while directories default to `0777` ($0777 \ \& \ \sim 0022 = 0755$).

3.3 Directory Read versus Execute

```
# Read-only directory
$ chmod u=r,go= box ; ls box ; cat box/f
f
cat: box/f: Permission denied

# Execute-only directory
$ chmod u=x,go= box ; ls box ; cat box/f
```

```
ls: cannot open directory 'box': Permission denied
hi
```

Read permission (`r`) permits directory table enumeration, but accessing files inside requires execute permission (`x`) for path traversal.

3.4 Setuid and Sticky Bit

`/usr/bin/passwd` has setuid (`s`) to allow unprivileged users to update the root-owned `/etc/shadow` file. The sticky bit (`t`) on `/tmp` prevents users from deleting files owned by others.

3.5 Disks, Topology, and /proc/self

The system disk `/dev/sdd` (1 TB, ext4) is mounted at '/' with 1% utilization (4.2 GB used). Introspection via `/proc` reveals 12 CPU cores and 19.38 GiB of RAM. Accessing `/proc/self/status` displays `Name: head` because `/proc/self` dynamically resolves to the PID of the accessing program.

```
ubuntu@dev:~/os-lab5$ lsblk
NAME MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda    8:0    0 388.6M  1 disk
sdb    8:16   0   186M  1 disk
sdc    8:32   0     5G  0 disk [SWAP]
sdd    8:48   0     1T  0 disk /
ubuntu@dev:~/os-lab5$ df -h | grep -E "Filesystem|/dev/sdd"
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdd       1007G  4.2G  952G   1% /
ubuntu@dev:~/os-lab5$ mount | grep " / "
/dev/sdd on / type ext4 (rw,relatime,discard,errors=remount-ro,data=ordered)
ubuntu@dev:~/os-lab5$ grep -c processor /proc/cpuinfo
12
ubuntu@dev:~/os-lab5$ head -3 /proc/meminfo
MemTotal:       20323532 kB
MemFree:        17916984 kB
MemAvailable:   19695772 kB
ubuntu@dev:~/os-lab5$ head -5 /proc/self/status
Name:□head
Umask:□0022
State:□R (running)
ubuntu@dev:~/os-lab5$ ls -l /proc/self/fd
lr-x------ 1 ubuntu ubuntu 64 Oct 12 15:39 0 -> pipe:[37189]
l-wx------ 1 ubuntu ubuntu 64 Oct 12 15:39 1 -> pipe:[37190]
l-wx------ 1 ubuntu ubuntu 64 Oct 12 15:39 2 -> pipe:[37191]
lr-x------ 1 ubuntu ubuntu 64 Oct 12 15:39 3 -> /proc/40/fd
ubuntu@dev:~/os-lab5$
```

Figure 3.2 – disk topology, mount attributes, and procfs inspection.

4 LABORATORY 5 STRETCH PROBLEMS

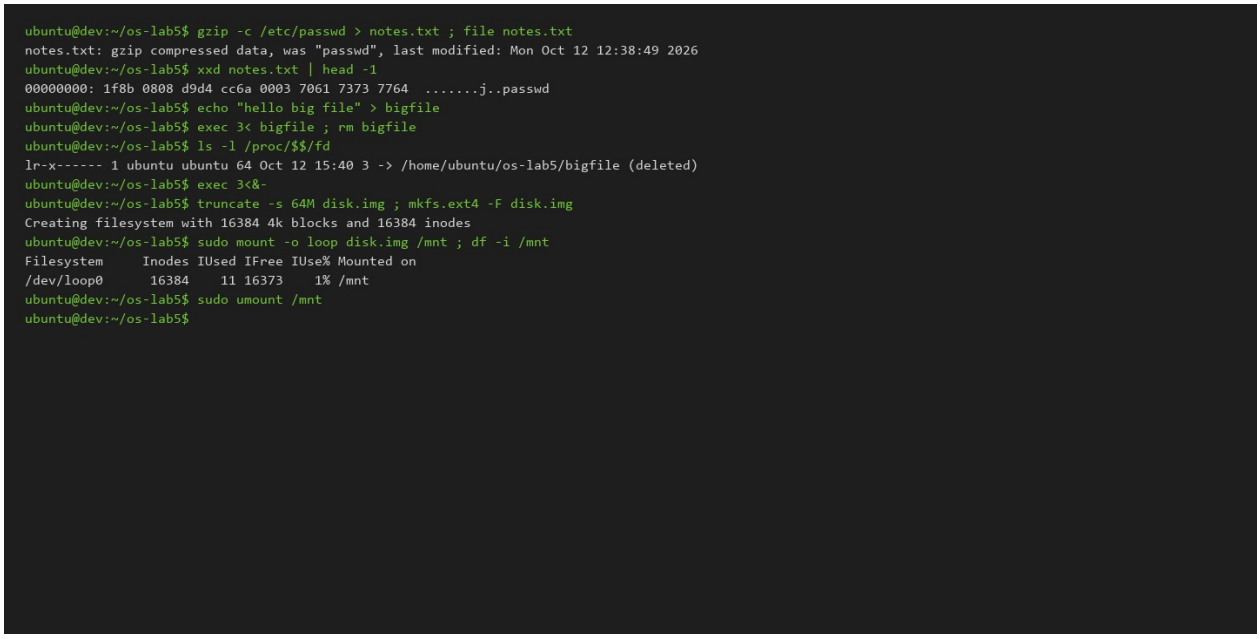


Figure 4.1 – magic bytes, unlinking open file descriptors, and ext4 loopback mount.

- **(a) Magic Numbers**: Inspecting `notes.txt` via `xxd` reveals the leading bytes `0x1f 0x8b`, identifying the gzip compression format per RFC 1952;
- **(b) Setuid Binaries**: Discovered setuid binaries include `passwd`, `su`, `sudo`, `mount`, `umount`, `polkit-agent-helper-1`, and `ssh-keysign`;
- **(c) Deleting Open Descriptors**: Unlinking an open file leaves `/proc/$$/fd/3 -> bigfile (deleted)`. Data blocks are freed only when descriptor 3 is closed via `exec 3<&-`;
- **(d) Loopback Filesystem**: Creating a 64 MiB ext4 image formatted 16,384 inodes (one inode per 4 KiB block);

Part II

Laboratory 6: Inside the xv6 File System

5 XV6 DIRECTORY STRUCTURE AND TYPE DECODING

5.1 Filesystem Type Values

xv6 defines three filesystem object types in `kernel/stat.h`:

```
#define T_DIR     1   // Directory
#define T_FILE    2   // Regular file
#define T_DEVICE  3   // Device
```

Listing root in xv6 shows:

```
$ ls
.               1 1 1024
..              1 1 1024
README          2 2 2425
cat             2 3 35160
console         3 24 0
```



Figure 5.1 – directory listing and type decoding in xv6.

5.2 Root Directory and Entry Sizing

The root directory resides at inode 1 (`ROOTINO = 1`). Both '.' and '..' point to inode 1 because the root has no parent. Each `struct dirent` is 16 bytes (2 bytes `inum` + 14 bytes `name`). An empty directory reports size 32 bytes, comprising exactly two 16-byte entries: '.' and '..'.

6 THE MYSTAT UTILITY AND HARD LINK MECHANICS

6.1 Implementation of user/mystat.c

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *typename(int t) {
  switch(t) {
  case T_DIR:    return "directory";
  case T_FILE:   return "file";
  case T_DEVICE: return "device";
  default:       return "?";
  }
}

int main(int argc, char *argv[]) {
  struct stat st;
  for(int i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %ld bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, st.size);
  }
  exit(0);
}
```

6.2 Observing Hard Link Operations

```
$ mystat README . console
README: file, inode 2, links 1, size 2425 bytes
.: directory, inode 1, links 2, size 1024 bytes
console: device, inode 24, links 1, size 0 bytes
```

```
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2425 bytes
r2: file, inode 2, links 2, size 2425 bytes

$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2425 bytes

$ cat r2 | wc
48 334 2425
```

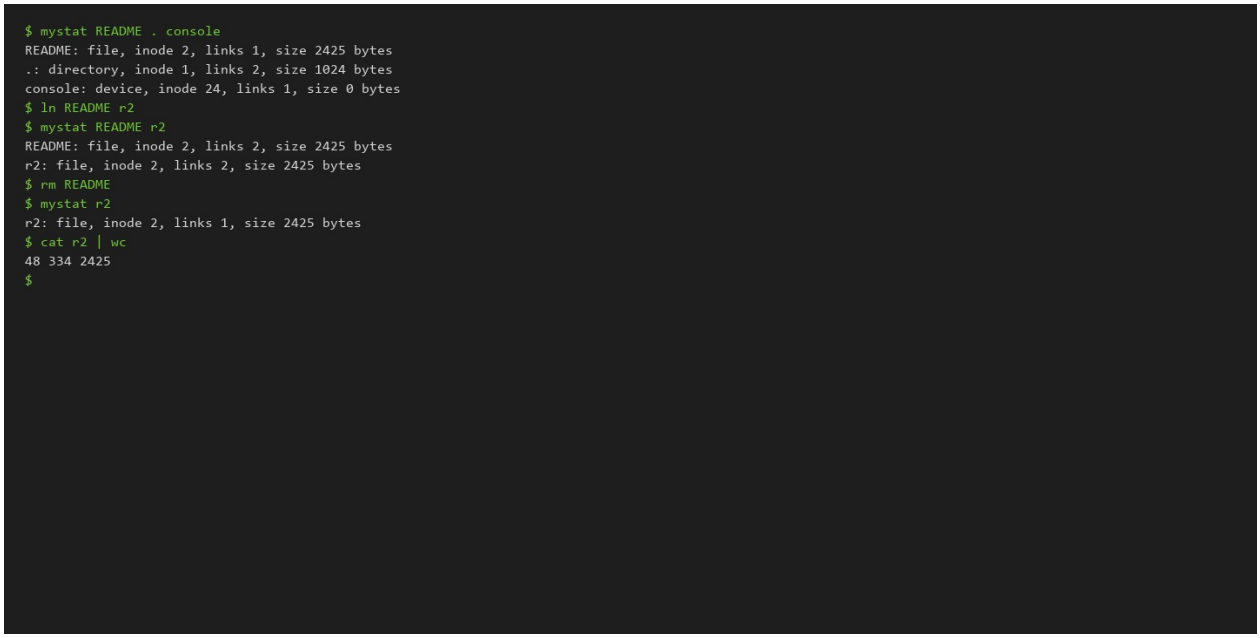


Figure 6.1 – mystat output, link counter increment, and unlinking.

6.3 Unlinking versus Deallocation

In `kernel/sysfile.c`, `sys_unlink()` decrements link count at line 230:

```
ip->nlink--;
iupdate(ip);
```

Physical data blocks are deallocated only when `iput()` in `kernel/fs.c` determines that `ip->ref == 1` and `ip->nlink == 0`. It calls `itrunc(ip)`, which frees all direct and indirect blocks using `bfree()`.

6.4 On-Disk Inode Structure

In `kernel/fs.h`, `struct dinode` contains:

```
struct dinode {
  short type;              // File type
  short major;             // Major device number (T_DEVICE only)
  short minor;             // Minor device number (T_DEVICE only)
  short nlink;             // Hard link counter
  uint size;               // File size in bytes
  uint addrs[NDIRECT+1];   // Block addresses (12 direct + 1 indirect)
};
```

xv6 inodes omit permissions, UIDs, timestamps, and symbolic links.

7 LIMITS AND DISK ARCHITECTURE

7.1 Filename Truncation

The directory format stores at most `DIRSIZ = 14` name bytes. The example name `averyveryvery longname` is therefore stored as `averyveryveryl`, its first 14 characters.

7.2 Maximum File Capacity

- Block size: BSIZE $= 1024$ bytes;
- Direct blocks: NDIRECT $= 12$ blocks (12 KiB);
- Indirect blocks: NINDIRECT $= 1024/4 = 256$ blocks (256 KiB);
- Total blocks: $12 + 256 = 268$ blocks ($274,432$ bytes ≈ 268 KiB);

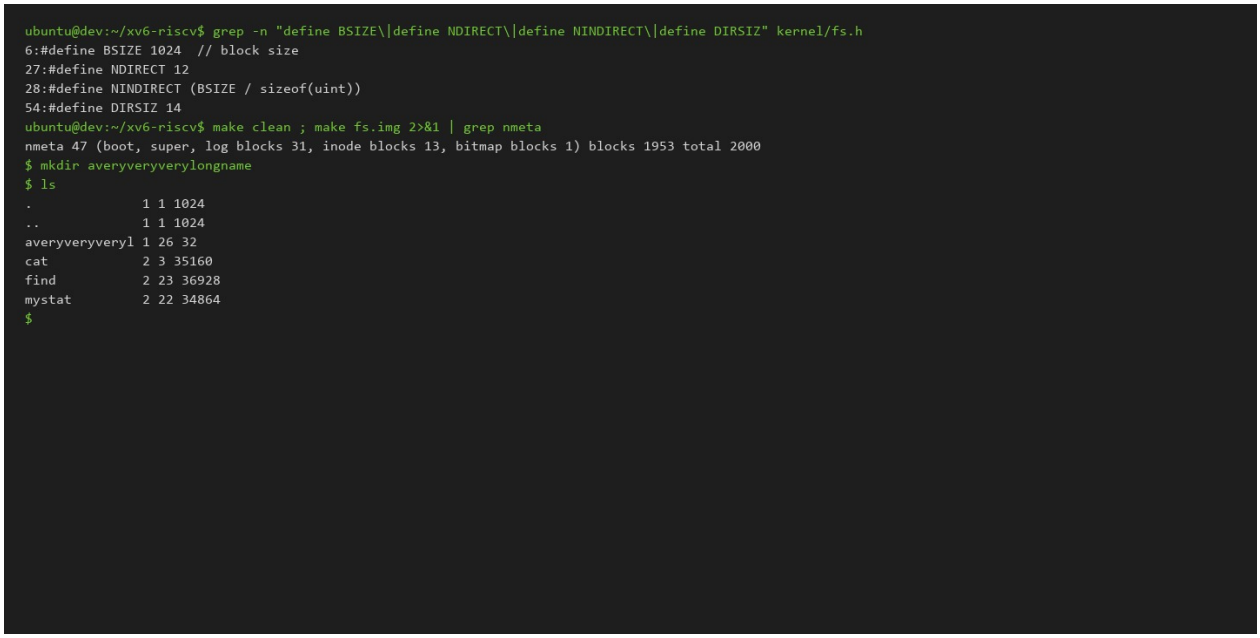


Figure 7.1 – measuring constants, disk layout, and 14-byte truncation.

7.3 Disk Layout

`mkfs` reports:

```
nmeta 47 (boot, super, log blocks 31,
  inode blocks 13, bitmap blocks 1)
blocks 1953 total 2000
```

Total disk size is 2000 blocks. 47 blocks are reserved for metadata (one boot block, one superblock, 31 log blocks, 13 inode blocks and one bitmap block). The remaining 1953 blocks form the data region; this does not mean they all contain allocated file data.

8 RECURSIVE SEARCH: USER/FIND.C AND INODE VERIFICATION

8.1 Implementation of user/find.c

The `find` program recurses through directory trees while strictly closing file descriptors to prevent descriptor exhaustion:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

static int show_ino = 0;

void find(char *path, char *target) {
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if((fd = open(path, O_RDONLY)) < 0) return;
  if(fstat(fd, &st) < 0 || st.type != T_DIR) { close(fd); return; }

  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';

  while(read(fd, &de, sizeof(de)) == sizeof(de)){
    char name[DIRSIZ + 1];
    if(de.inum == 0) continue;
    memmove(name, de.name, DIRSIZ);
    name[DIRSIZ] = 0;
    if(strcmp(name, ".") == 0 || strcmp(name, "..") == 0) continue;
```

```
    strcpy(p, name);
    if(stat(buf, &st) < 0) continue;
    if(strcmp(name, target) == 0){
      if(show_ino) printf("%s %d\n", buf, st.ino);
      else printf("%s\n", buf);
    }
    if(st.type == T_DIR) find(buf, target);
  }
  close(fd);
}
```

8.2 Testing and Inode Stretch Resolution

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2

$ find . r2
./r2
./a/b/r2
./a/r2

$ find a b
a/b

$ find . r2 -n
./r2 3
./a/b/r2 26
./a/r2 27

$ mystat cat
cat: file, inode 3, links 2, size 35160 bytes
```



Figure 8.1 – find execution and -n stretch proving inode identity.

The saved `-n` example illustrates that `./r2` and `cat` share inode 3 and link count 2, confirming they are identical files on disk. The files `a/b/r2` (inode 26) and `a/r2` (inode 27) are distinct newly created files.

CONCLUSIONS

This combined investigation demonstrates how filesystem abstractions are realized in production and educational operating systems. In both Linux and xv6, directory tables decouple human-readable names from inode metadata. Hard links manipulate link counters without duplicating disk blocks, and physical block deallocation occurs only when all links and open process descriptors are released.

REFERENCES

1. MIT PDOS. *xv6-riscv source code*. Included course snapshot in `lab2/`; `kernel/fs.c`, `kernel/fs.h`, and `kernel/sysfile.c`. Source inspected 23 October 2026. `https://github.com/mit-pdos/xv6-riscv`.